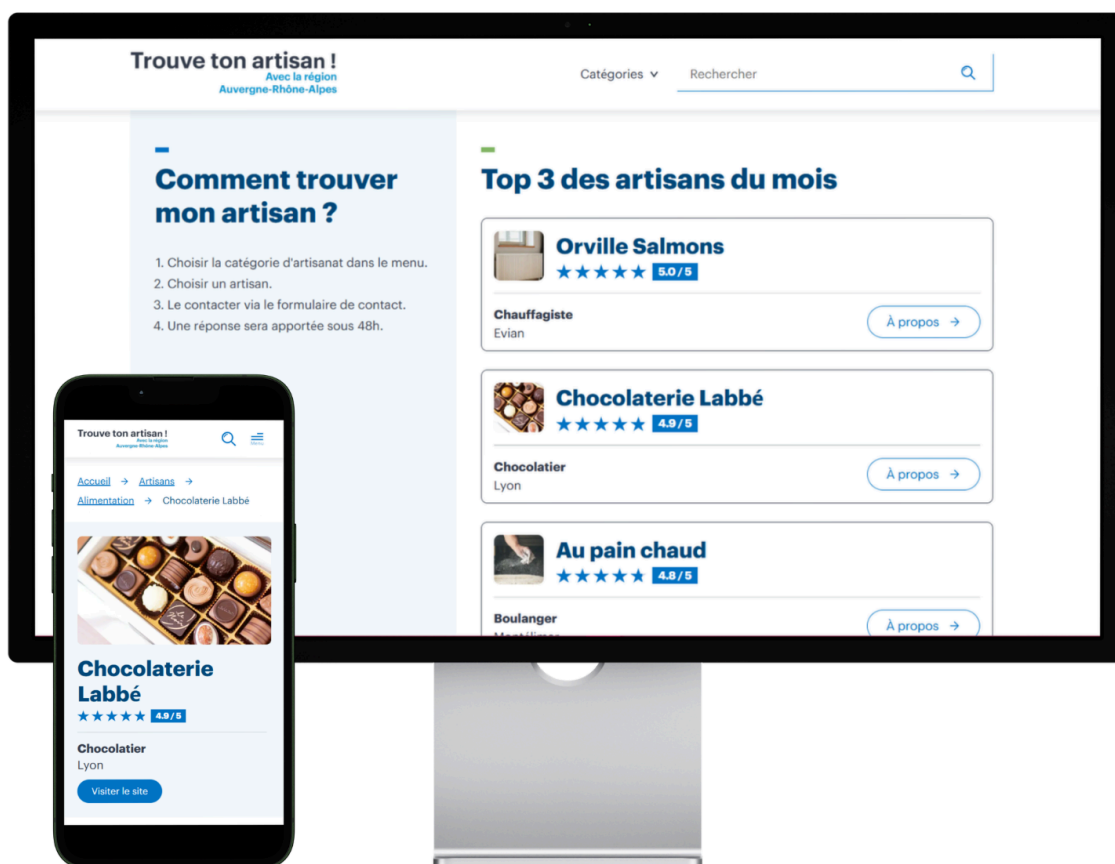


Projet : Trouve ton artisan

Devoir Bilan :

Créez le site Web "Trouve ton artisan" avec React.JS

Auteur	Cédric Kernec
GitHub	https://github.com/pixseed
Formation	Développeur Web & Web Mobile - Centre Européen de Formation
Technologies	React, React Router, Vite, Node.js, Express, Sequelize, MySQL
Date	06/2026
Version	1.0.0
Site en ligne	https://auvergnerhonealpes-trouve-ton-artisan.netlify.app/



Sommaire

- **Projet : Trouve ton artisan**
 - Sommaire
 - 1. Contexte du projet
 - 2. Présentation du client
 - 3. Expression des besoins
 - 4. Contraintes techniques
 - 5. Fonctionnalités
 - 5.1. Navigation (pages)
 - 5.2. Header
 - 5.3. Footer
 - 5.4. Page d'accueil
 - 5.5. Page liste des artisans
 - 5.6. Page fiche artisan
 - 5.7. Fonctionnalités avancées
 - 6. Architecture de l'application
 - 7. Livrables attendus
 - 8. Maquettes et wireframes
 - 8.1. Wireframes
 - 8.2. Maquettes
 - 9. Base de données
 - 9.1. Modèle Conceptuel de Données (MCD)
 - 9.2. Modèle Logique de Données (MLD)
 - 9.3. Script MySQL
 - 10. Déploiement
 - 10.1. Architecture de déploiement
 - 10.2. Variables d'environnement
 - 11. Veille sécurité
 - 11.1 Méthodologie de veille
 - 11.2. Vulnérabilités étudiées
 - 11.2.1. Entrées utilisateur non fiables
 - 11.2.2. Exposition excessive des erreurs serveur
 - 11.2.3. Abus d'API / spam / surcharge
 - 11.2.4. Requêtes cross-origin non maîtrisées
 - 11.2.5. Headers HTTP insuffisants
 - 11.3. Arbitrages réalisés dans le projet
 - 12. Sécurité mise en place
 - 12.1. Sécurisation des headers HTTP (Helmet)
 - 12.2. Contrôle des requêtes cross-origin (CORS)
 - 12.3. Limitation de la taille des requêtes JSON envoyées
 - 12.4. Validation des données côté serveur
 - 12.5. Nettoyage des données utilisateur (Sanitization)
 - 12.6. Protection contre l'abus de requêtes (Rate Limiting)
 - 12.6.1. Protection globale de l'API
 - 12.6.2. Protection du formulaire de contact
 - 12.7. Gestion centralisée des erreurs API
 - 12.7.1. Classe métier `ApiError`

- 12.7.2. Middleware global de gestion des erreurs
- 12.7.3. Protection des informations sensibles
- 12.7.4. Cohérence frontend / backend
- 12.7.5. Séparation des responsabilités
- 13. Qualité du projet
 - 13.1. Validation W3C
 - 13.2. Audit Lighthouse
- 14. Liens du projet

1. Contexte du projet

La région Auvergne-Rhône-Alpes souhaite proposer une plateforme permettant aux particuliers de trouver facilement un artisan local et d'entrer en contact avec lui via un formulaire.

Le projet doit respecter plusieurs contraintes :

- Interface responsive design
 - Navigation simple
 - Accessibilité WCAG
 - Intégration avec une API backend
 - Base de données contenant les artisans et les catégories
 - Architecture sécurisée
 - Hébergement en ligne
-

2. Présentation du client

La région Auvergne-Rhône-Alpes souhaite mettre à disposition un service numérique permettant de valoriser les artisans locaux et faciliter leur mise en relation avec les particuliers.

Site institutionnel : <https://www.auvergnerhonealpes.fr/>

Trouve ton artisan !
Avec la région
Auvergne-Rhône-Alpes

3. Expression des besoins

L'application doit permettre de :

- Consulter une liste d'artisans
 - Filtrer les artisans par catégorie
 - Rechercher par nom
 - Consulter la fiche détaillée d'un artisan
 - Contacteur un artisan via un formulaire
-

4. Contraintes techniques

Architecture technique du projet :

Frontend

- React
- Vite
- React Router
- Bootstrap
- SCSS

Backend

- Node.js
- Express
- Sequelize

Base de données

- MySQL



5. Fonctionnalités

5.1. Navigation (pages)

- Page d'accueil
- Liste des artisans (par catégorie)
- Fiche artisan
- Page 404
- Pages Légales (en cours de construction)

5.2. Header

- Logo
- Menu catégories
- Barre de recherche

5.3. Footer

Liens pages légales

Informations de contact

5.4. Page d'accueil

Présentation du fonctionnement du site en 4 étapes

Affichage des 3 artisans du mois

5.5. Page liste des artisans

Filtrage par catégorie

Affichage sous forme de cards

Accès à la fiche détaillée

5.6. Page fiche artisan

Informations affichées :

Nom

Image

Note

Spécialité

Localisation

Site web éventuel

Description

Galerie d'images

Formulaire de contact

5.7. Fonctionnalités avancées

Chargement dynamique des artisans depuis l'API

Gestion des erreurs API

Filtrage par catégorie

Recherche par nom / ville / spécialité

Galerie d'images responsive avec ouverture plein écran

Formulaire de contact sécurisé

Breadcrumb de navigation

6. Architecture de l'application

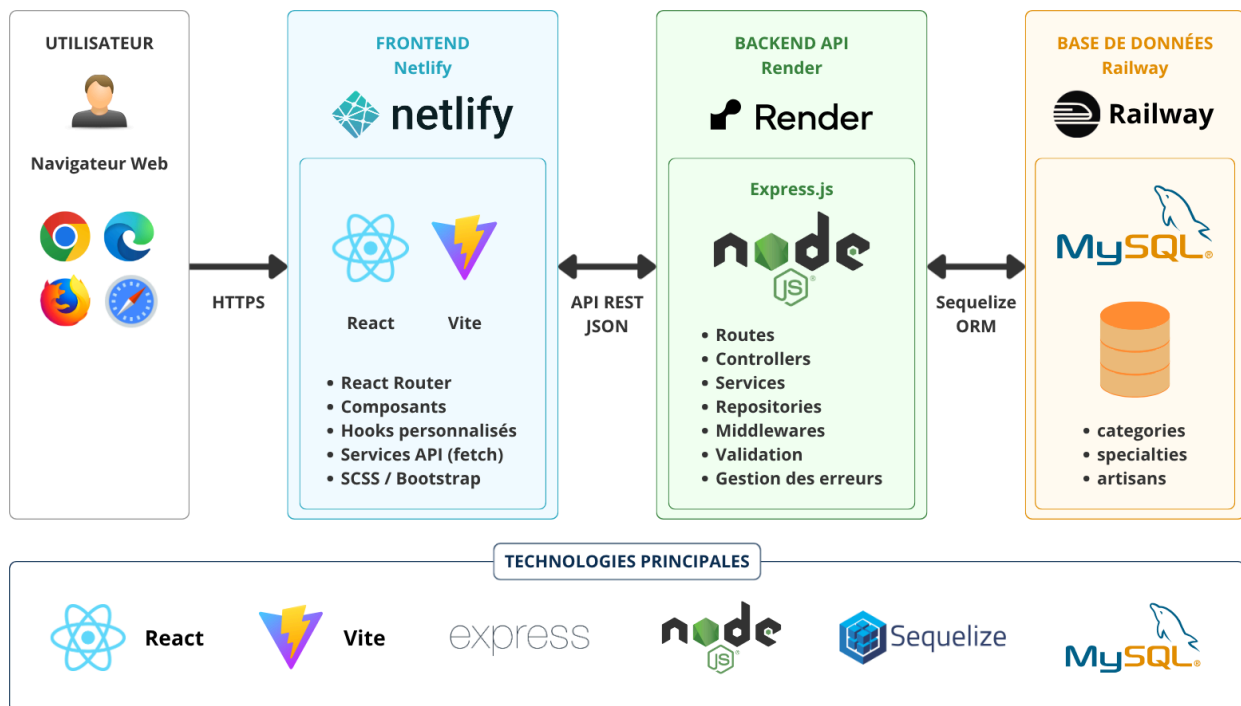
Routes frontend :

```

/                → page accueil
/artisans        → liste des artisans
/artisans/:id    → fiche artisan
  
```

1. ARCHITECTURE GLOBALE DE L'APPLICATION

Vue d'ensemble des composants et des déploiements



Le frontend React communique avec une **API REST** développée avec **Express**.

Les données sont récupérées depuis une base **MySQL** via l'**ORM Sequelize**.

L'application est déployée sur plusieurs services afin de séparer :

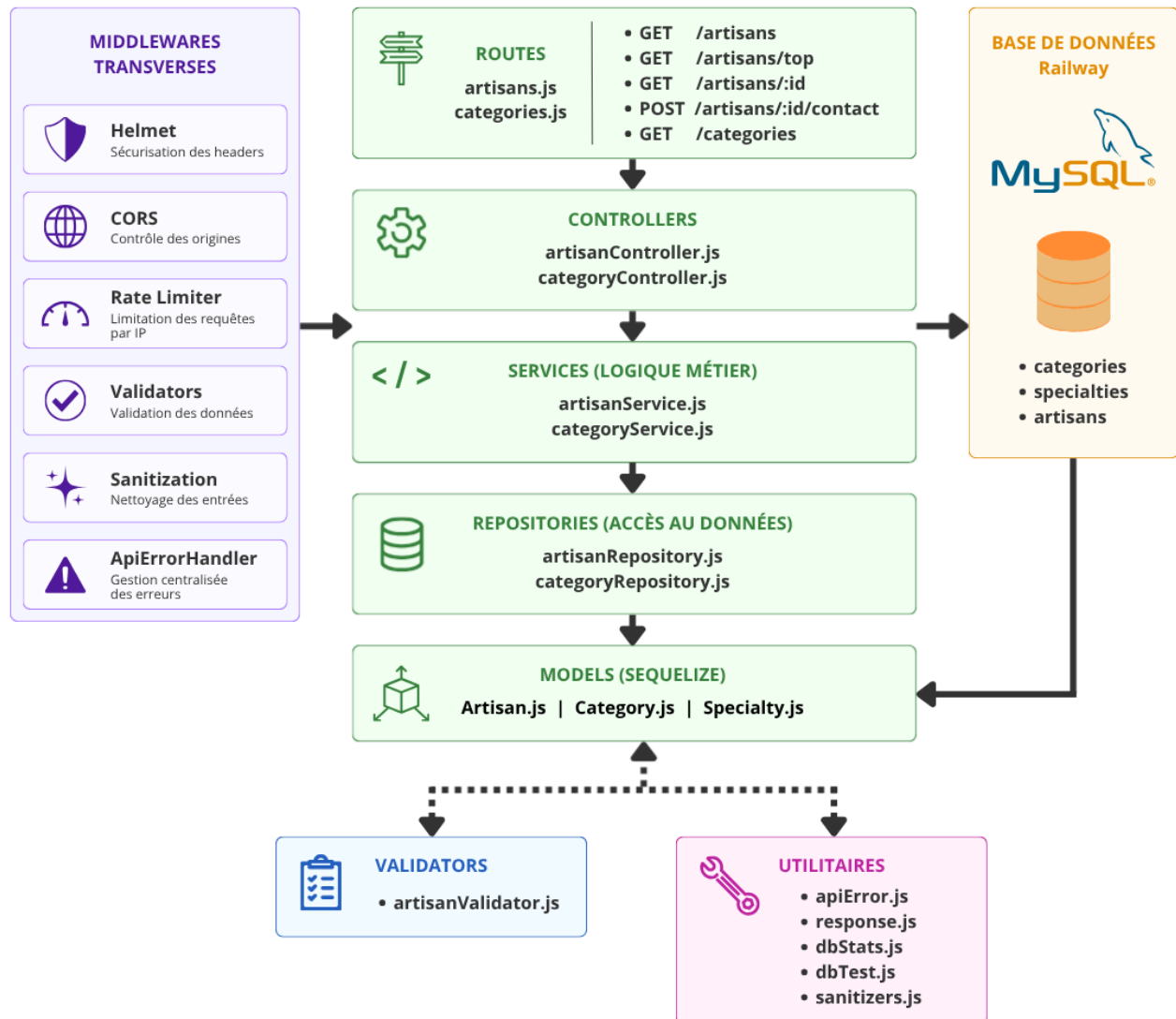
l'interface utilisateur

la logique métier

et le stockage de données

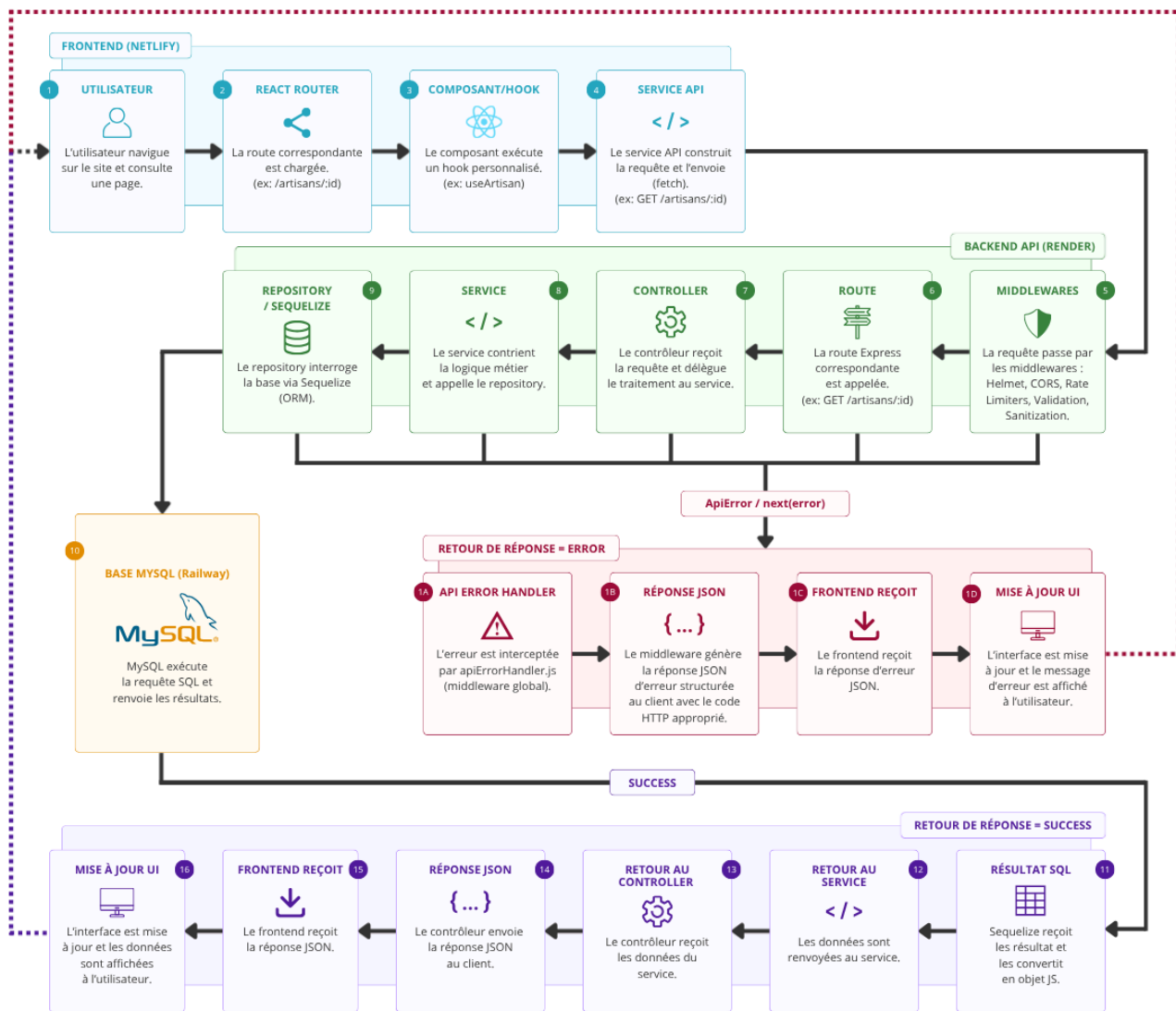
2. ARCHITECTURE BACKEND (LOGIQUE)

Organisation interne de l'API Express



3. FLUX D'UNE REQUÊTE API - DU NAVIGATEUR À LA RÉPONSE

Exemple : Consultation d'une fiche artisan



7. Livrables attendus

Wireframes (Desktop, Tablet, Mobile)

API Node.js fonctionnelle

Base de données MySQL

Frontend React

Interface responsive

Documentation du projet

8. Maquettes et wireframes

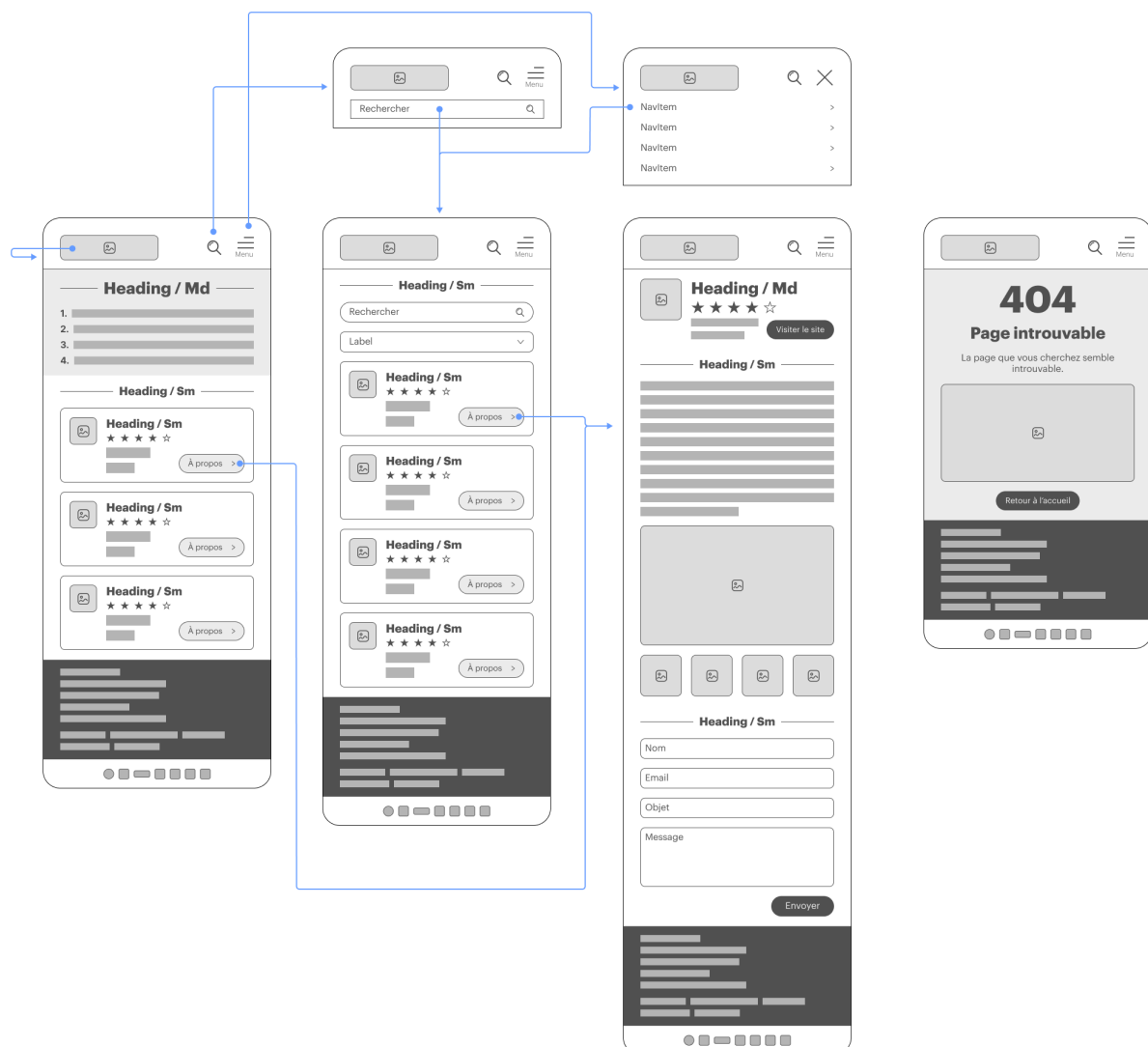
LIEN VERS LES MAQUETTES FIGMA

Voir les maquettes Figma :

https://www.figma.com/design/C0moU99nW9cfFIHHRzYXxc/Kernec_Cedric_Devoir_Bilan_Trouve_Ton_Artisan?node-id=38-3103&t=fcY6xDtTEQbigvnm-1

8.1. Wireframes

WIREFRAME MOBILE



WIREFRAME TABLET

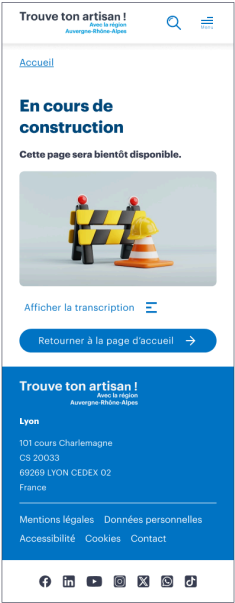
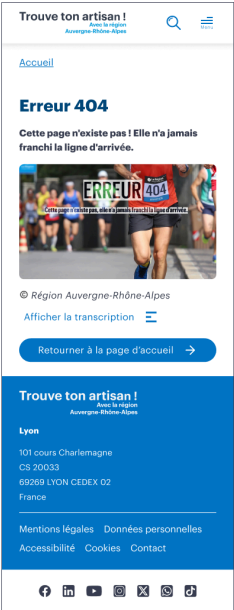
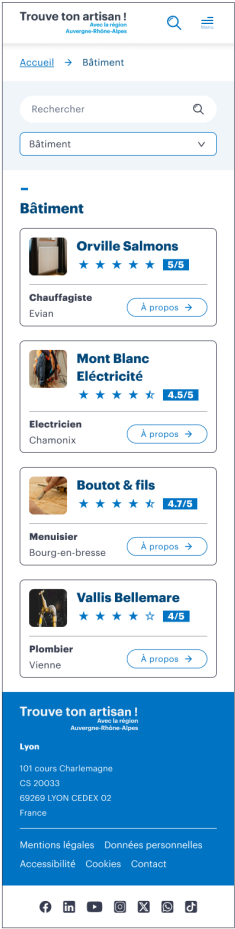
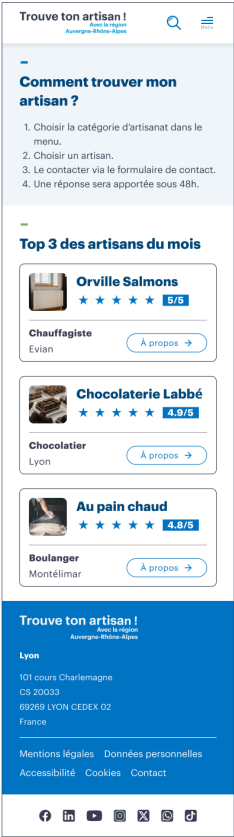


WIREFRAME DESKTOP



8.2. Maquettes

MAQUETTE MOBILE



MAQUETTE TABLET

Trouve ton artisan !
Région Auvergne-Rhône-Alpes

Catégories ▼

Rechercher

Q

Comment trouver mon artisan ?

1. Choisir la catégorie d'artisanat dans le menu.

2. Choisir un artisan.

3. Le contacter via le formulaire de contact.

4. Une réponse sera apportée sous 48h.

Top 3 des artisans du mois

Orville Salmons
★★★★★ 5/5
Chauffagiste
Evian
À propos →

Chocolaterie Labbé
★★★★★ 4.9/5
Chocolatier
Lyon
À propos →

Au pain chaud
★★★★★ 4.9/5
Boulangier
Montélimar
À propos →

Trouve ton artisan !
Région Auvergne-Rhône-Alpes

Lyon
101 cours Charlemagne
CS 20033
69069 LYON CEDEX 02
France

Mentions légales
Données personnelles
Accessibilité
Cookies
Contact

f

in

u

o

o

o

o

Trouve ton artisan !
Région Auvergne-Rhône-Alpes

Catégories ▼

Rechercher

Q

Accueil → Bâtiment

Rechercher

Q

Bâtiment ▼

Bâtiment

Orville Salmons
★★★★★ 5/5
Chauffagiste
Evian
À propos →

Mont Blanc Électricité
★★★★★ 4.5/5
Electricien
Chamonix
À propos →

Boutot & fils
★★★★★ 4.7/5
Menuisier
Bourg-en-bresse
À propos →

Vallis Bellemare
★★★★★ 4/5
Plombier
Vienne
À propos →

Trouve ton artisan !
Région Auvergne-Rhône-Alpes

Lyon
101 cours Charlemagne
CS 20033
69069 LYON CEDEX 02
France

Mentions légales
Données personnelles
Accessibilité
Cookies
Contact

f

in

u

o

o

o

o

Trouve ton artisan !
Région Auvergne-Rhône-Alpes

Catégories ▼

Rechercher

Q

Accueil → Bâtiment → Orville Salmons

Orville Salmons
★★★★★ 5/5
Chauffagiste
Evian
Visiter le site

À propos

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Phasellus eleifend ante sem, id volutpat massa fermentum nec. Praesent volutpat scelerisque mauris, quis sollicitudin tellus sollicitudin.

Galerie

Contact

Nom

Email

Objet

Message

Envoyer

Trouve ton artisan !
Région Auvergne-Rhône-Alpes

Lyon
101 cours Charlemagne
CS 20033
69069 LYON CEDEX 02
France

Mentions légales
Données personnelles
Accessibilité
Cookies
Contact

f

in

u

o

o

o

o

Trouve ton artisan !
Région Auvergne-Rhône-Alpes

Catégories ▼

Rechercher

Q

Accueil

Erreur 404

Cette page n'existe pas ! Elle n'a jamais franchi la ligne d'arrivée.

© Région Auvergne-Rhône-Alpes
Afficher la transcription

Retourner à la page d'accueil →

Trouve ton artisan !
Région Auvergne-Rhône-Alpes

Lyon
101 cours Charlemagne
CS 20033
69069 LYON CEDEX 02
France

Mentions légales
Données personnelles
Accessibilité
Cookies
Contact

f

in

u

o

o

o

o

Trouve ton artisan !
Région Auvergne-Rhône-Alpes

Catégories ▼

Rechercher

Q

Accueil

En cours de construction

Cette page sera bientôt disponible.

Afficher la transcription

Retourner à la page d'accueil →

Trouve ton artisan !
Région Auvergne-Rhône-Alpes

Lyon
101 cours Charlemagne
CS 20033
69069 LYON CEDEX 02
France

Mentions légales
Données personnelles
Accessibilité
Cookies
Contact

f

in

u

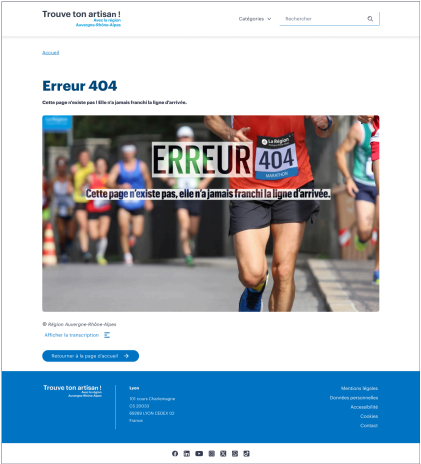
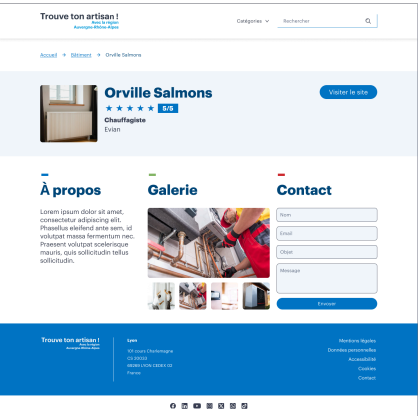
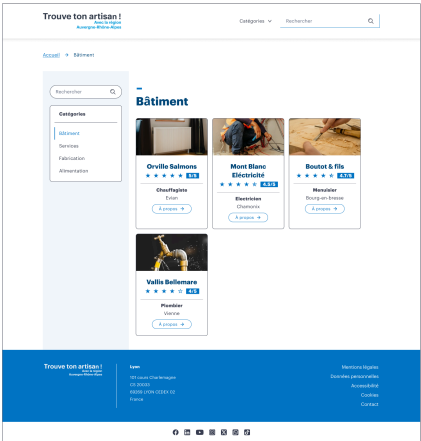
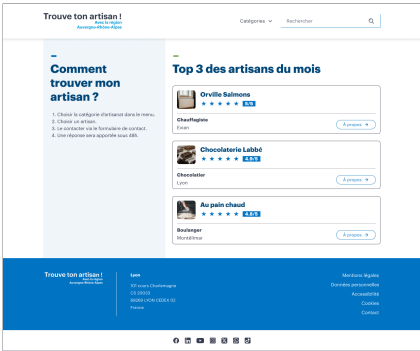
o

o

o

o

MAQUETTE DESKTOP



9. Base de données

Présentation de la base de données permettant de gérer un annuaire d'artisans.

La base de données a été conçue sur une **logique de modélisation en deux étapes** :

un Modèle Conceptuel de Données (MCD)

un Modèle Logique de Données (MLD)

9.1. Modèle Conceptuel de Données (MCD)

Le **MCD** permet de représenter les entités métier et leurs relations avant la phase de développement.

Il met en évidence :

Les catégories d'artisans

Les spécialités associées

Les artisans

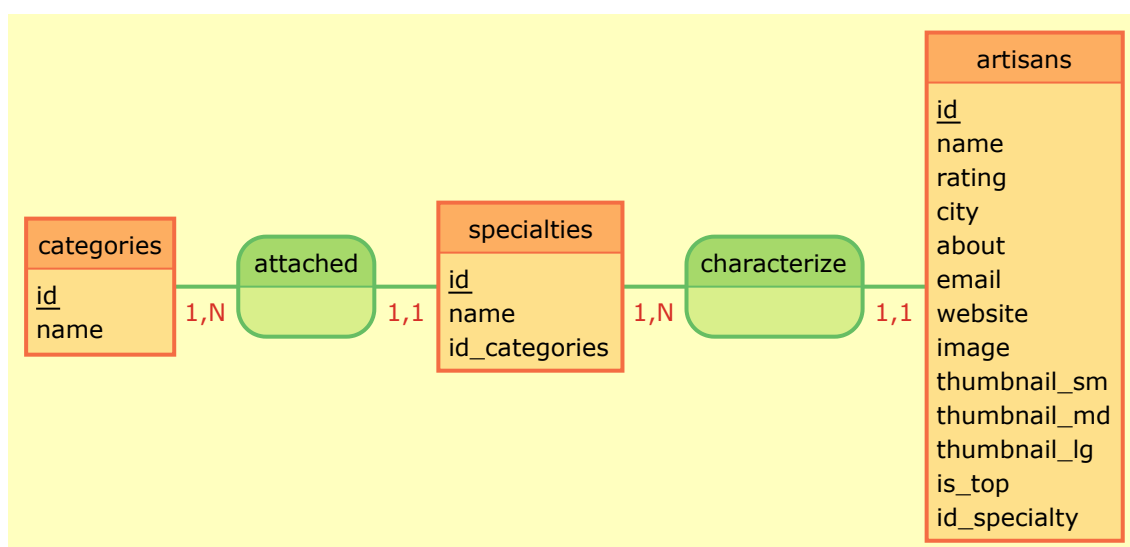
Relations :

Une catégorie peut être attachée à plusieurs spécialités (1,N)

Une spécialité est attachée à une seule catégorie (1,1)

Une spécialité peut caractériser plusieurs artisans (1,N)

Un artisan est caractérisé par une seule spécialité (1,1)



9.2. Modèle Logique de Données (MLD)

Le **MLD** traduit le MCD en structure relationnelle exploitable en base de données.

Il définit :

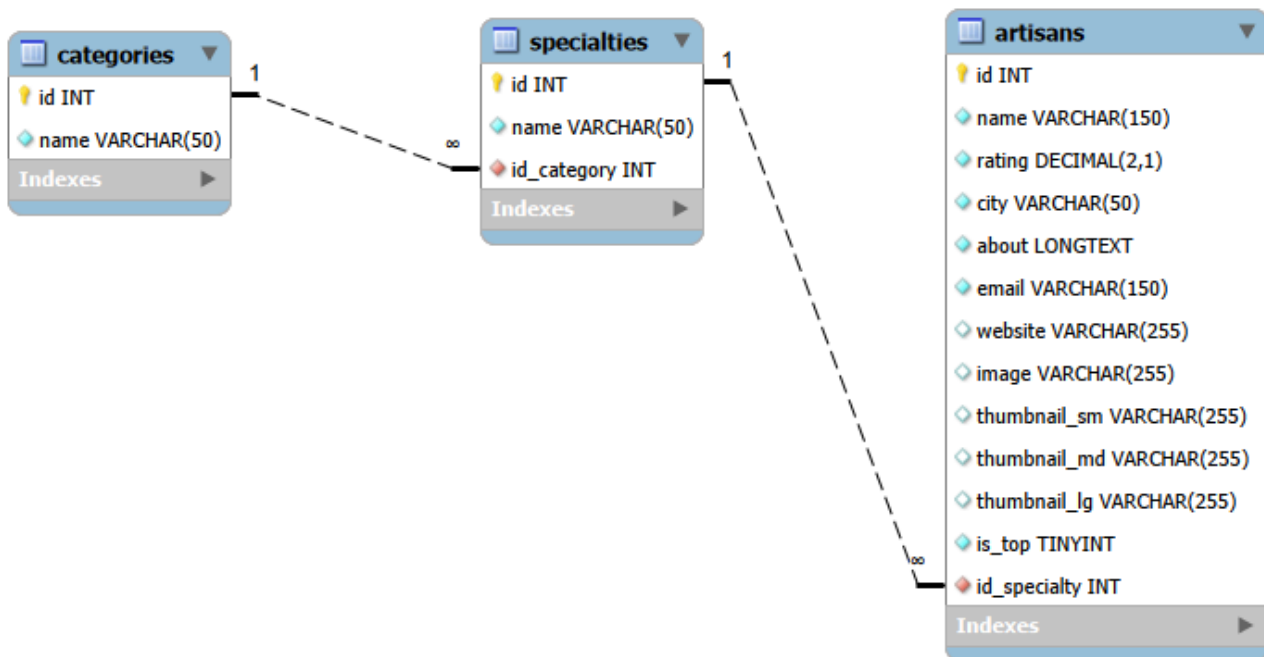
Tables	Clés primaires	Clés étrangères	Contraintes
categories specialties artisans	PK	FK	Relations Intégrité

Relations :

```
specialties.id_category → categories.id
```

```
artisans.id_specialty → specialties.id
```

Ce modèle est directement utilisé pour la création de la base de données en SQL.



9.3. Script MySQL

Des requêtes SQL ont été mises en place afin de répondre aux fonctionnalités principales du projet :

Recherche d'artisans	Filtrage par catégorie	Top des artisans du mois
----------------------	------------------------	--------------------------

Exemples :

RÉCUPÉRER LES 3 ARTISANS DU MOIS

```
SELECT
    a.name AS artisan,
    a.rating,
    s.name AS specialty,
    a.city
FROM artisans a
JOIN specialties s ON a.id_specialty = s.id
WHERE a.is_top = TRUE
ORDER BY a.rating DESC
LIMIT 3;
```

RÉCUPÉRER LES ARTISANS D'UNE CATÉGORIE

```
SELECT
    a.name AS artisan,
    a.rating,
    s.name AS specialty,
    a.city,
    c.name AS category
FROM artisans a
JOIN specialties s ON a.id_specialty = s.id
JOIN categories c ON s.id_category = c.id
WHERE c.id = 1
ORDER BY a.rating DESC;
```

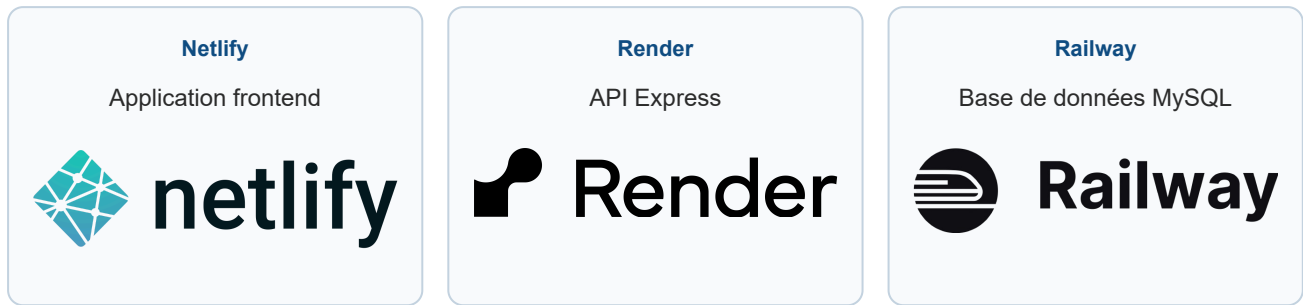
RÉCUPÉRER LES ARTISANS SUITE À UNE RECHERCHE PAR SAISIE

```
SELECT
    a.name AS artisan,
    a.rating,
    s.name AS specialty,
    a.city,
    c.name AS category
FROM artisans a
JOIN specialties s ON a.id_specialty = s.id
JOIN categories c ON s.id_category = c.id
WHERE
    a.name LIKE '%au%'
    OR s.name LIKE '%au%'
    OR a.city LIKE '%au%'
ORDER BY a.rating DESC;
```

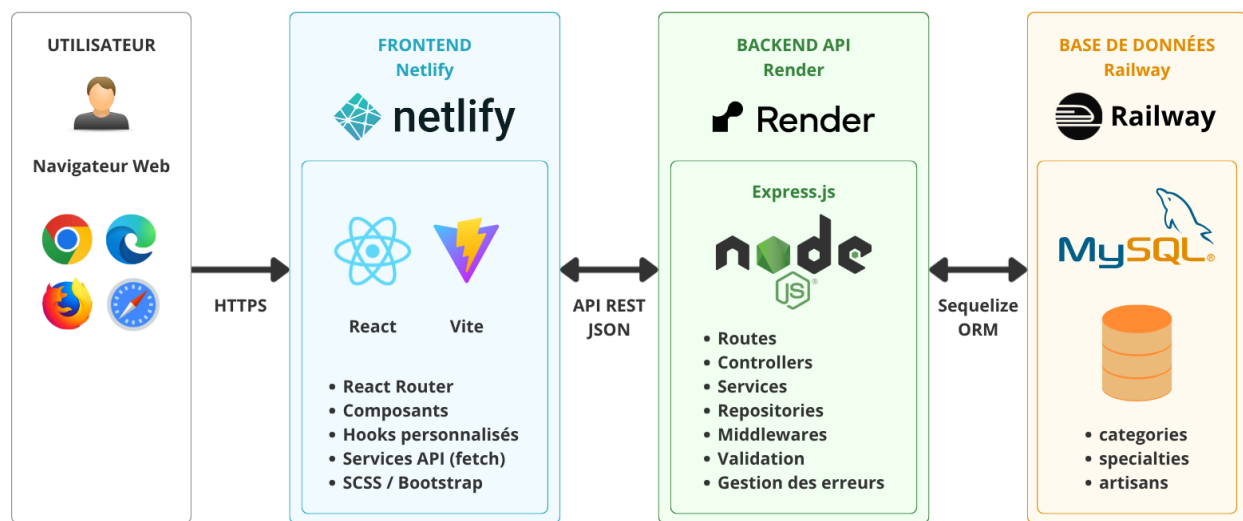
Ces requêtes ont été testées et validées.

10. Déploiement

Le projet est déployé sur **3 services distincts** :



10.1. Architecture de déploiement



10.2. Variables d'environnement

Des variables d'environnement ont été utilisées **afin de sécuriser la configuration du projet** :

URL de l'API

URL du frontend autorisé (CORS)

Informations de connexion à la base de données

Ces informations sensibles ne sont jamais stockées dans le dépôt GitHub.

Cette architecture permet :

Une meilleure séparation des responsabilités

Une maintenance simplifiée

Un hébergement indépendant

11. Veille sécurité

11.1 Méthodologie de veille

Dans le cadre du développement du projet, une veille de sécurité a été menée afin d'identifier les vulnérabilités courantes susceptibles d'affecter une application web exposant une API backend.

OBJECTIF

Anticiper les risques de sécurité et de vulnérabilité applicable à l'architecture du projet (frontend React + API Express + base de données MySQL).

Sources : Les principales sources consultées ont été :

OWASP Top 10	https://owasp.org/www-project-top-ten/
Documentation officielle Express	https://expressjs.com/
Documentation officielle Helmet	https://github.com/helmetjs/helmet
Documentation MDN Web Docs	https://developer.mozilla.org/
Documentation express-rate-limit	https://github.com/express-rate-limit/express-rate-limit

Cette veille a permis d'identifier plusieurs risques concrets liés aux entrées utilisateur, aux erreurs serveur, aux abus d'API ainsi qu'aux politiques de sécurité navigateur.

11.2. Vulnérabilités étudiées

11.2.1. Entrées utilisateur non fiables

Risques :

Données invalides

Injectons

Corruption

11.2.2. Exposition excessive des erreurs serveur

Risques :

Fuite stack trace

Structure backend exposée

11.2.3. Abus d'API / spam / surcharge

Risques :

Spam formulaire

Surcharges des ressources serveur

11.2.4. Requêtes cross-origin non maîtrisées

Risques :

Appels navigateur non souhaités

11.2.5. Headers HTTP insuffisants

Risques :

XSS (Cross-Site Scripting)

Vulnérabilité permettant d'injecter du code malveillant dans une application web afin qu'il soit exécuté par le navigateur.

Clickjacking

Technique consistant à piéger un utilisateur intégrant l'application dans un cadre invisible (``iframe``) ou trompeur afin de détourner ses clics.

11.3. Arbitrages réalisés dans le projet

Cette veille a influencé les choix d'architecture présentés dans la section 11. **Sécurité mise en place**, notamment :

validation systématique côté backend

sanitization des entrées utilisateurs

limitation de débit des requêtes

sécurisation des headers HTTP

gestion centralisée des erreurs

masquage des informations sensibles en production

12. Sécurité mise en place

12.1. Sécurisation des headers HTTP (Helmet)

Afin de **renforcer la sécurité globale de l'API backend**, le middleware **Helmet** a été mis en place.

Helmet ajoute automatiquement plusieurs en-têtes HTTP de sécurité permettant de limiter certaines attaques côté navigateur.

Cela permet de :

Réduire le risque XSS*

Limiter le risque de clickjacking**

Une meilleure sécurité navigateur

Masquer les informations techniques

*XSS (Cross-Site Scripting)

Vulnérabilité permettant à un utilisateur malveillant d'injecter du code dans une application web afin qu'il soit exécuté par le navigateur.

*Clickjacking

Technique consistant à piéger un utilisateur intégrant l'application dans un cadre invisible (`iframe`) ou trompeur afin de détourner ses clics.

CONFIGURATION SPÉCIFIQUE

```
crossOriginResourcePolicy: { policy: "cross-origin" }
```

Cette configuration permet l'accès contrôlé aux ressources statiques (notamment les images des artisans) depuis le frontend React hébergé sur une origine différente du backend API.

Sans cet ajustement, certaines ressources auraient été bloquées par le navigateur.

12.2. Contrôle des requêtes cross-origin (CORS)

Le backend expose une API consommée par une application frontend React. Par défaut, les navigateurs bloquent les requêtes entre origines différentes pour des raisons de sécurité.

Le middleware `CORS` a donc été configuré **afin d'autoriser uniquement le frontend officiel**.

Cela permet de :

Limiter les requêtes cross-origin autorisées depuis un navigateur

Centraliser la configuration selon l'environnement (development/production)

CONFIGURATION CORS

```
origin: process.env.FRONTEND_URL
```

Cette configuration autorise uniquement le frontend officiel à consommer l'API depuis le navigateur.

L'utilisation d'une variable d'environnement permet d'adapter l'URL selon l'environnement sans l'écrire directement dans le code source.

12.3. Limitation de la taille des requêtes JSON envoyées

Le backend accepte des données JSON provenant du frontend, notamment lors de l'envoi du formulaire de contact.

Afin d'éviter les abus ou l'envoi de charges excessives, une limite de taille a été définie.

Cela permet de :

Limiter la surcharge mémoire

Bloquer les contenus anormalement volumineux

Réduire le risque de déni de service (DoS)*

*Denial of Service (DoS)

Attaque visant à saturer un serveur par un très grand nombre de requêtes afin de perturber ou empêcher son fonctionnement normal.

LIMITATION DES REQUÊTES JSON

```
express.json({ limit: "10kb" })
```

Cette limite empêche l'envoi de charges JSON trop volumineuses vers l'API.

Dans ce projet, 10kb est suffisant pour les données attendues du formulaire de contact.

12.4. Validation des données côté serveur

Même si le frontend applique déjà des contrôles sur le formulaire de contact, **aucune validation côté client ne doit être considérée comme suffisante en matière de sécurité**. Un utilisateur malveillant peut contourner totalement l'interface frontend et envoyer directement des requêtes vers l'API.

Pour cette raison, **une validation complète est systématiquement effectuée côté backend**.

Les champs contrôlés sont :

Nom

Adresse email

Objet du message

Contenu du message

Le backend vérifie notamment :

La présence des champs
obligatoires

Le format de l'adresse email

La conformité des valeurs
attendues

En cas d'erreur une réponse structurée est retournée :

```
{
  "success": false,
  "error": {
    "message": "Données invalides.",
    "code": "VALIDATION_ERROR",
    "fields": {
      "email": "Adresse email invalide"
    }
  }
}
```

Cela garantit que seules des données valides peuvent être traitées par l'application.

12.5. Nettoyage des données utilisateur (Sanitization)

Avant validation et traitement, les données envoyées par l'utilisateur sont nettoyées via une étape de **sanitization**.

Cette étape permet de :

supprimer des espaces inutiles

neutraliser des caractères spéciaux potentiellement interprétables par le navigateur

normaliser des chaînes utilisateur avant validation

Cette pratique permet de :

Éviter les incohérences de
données

Fiabiliser la validation

Réduire certains comportements
inattendus liés aux entrées
utilisateurs

12.6. Protection contre l'abus de requêtes (Rate Limiting)

Le backend met en place une limitation du nombre de requêtes afin de protéger l'API contre les usages abusifs.

Deux niveaux de protection ont été implémentés à l'aide du middleware `express-rate-limit`.

12.6.1. Protection globale de l'API

Une limitation générale est appliquée à l'ensemble des routes de l'API.

Principe :

maximum 300 requêtes

sur une période de 15 minutes

par adresse IP

Cette protection permet de :

Limiter les usages abusifs
involontaires ou automatisés

Réduire les risques de surcharge
serveur

Éviter une consommation
excessive des ressources
backend

Un retour explicite est envoyé en cas de dépassement :

```
{
  "success": false,
  "error": {
    "message": "Trop de requêtes exécutées. Veuillez réessayer ultérieurement.",
    "code": "RATE_LIMIT_EXCEEDED"
  }
}
```

12.6.2. Protection du formulaire de contact

Le formulaire de contact constitue un point d'entrée plus sensible car il peut être ciblé pour du spam automatisé.

Une limitation plus stricte a donc été appliquée :

maximum 10 tentatives

sur une période de 15 minutes

par adresse IP

Cette protection permet de limiter :

Le spam automatisé

Les envois massifs abusifs

Les tentatives de surcharge de
l'API

Un retour explicite est envoyé en cas de dépassement :

```
{
  "success": false,
  "error": {
    "message": "Trop de tentatives d'envoi du formulaire. Veuillez réessayer ultérieurement.",
    "code": "CONTACT_RATE_LIMIT_EXCEEDED"
  }
}
```

12.7. Gestion centralisée des erreurs API

Une architecture centralisée de gestion des erreurs a été mise en place **afin d'assurer des réponses cohérentes, sécurisées et facilement exploitables par le frontend**.

Sans ce mécanisme, chaque contrôleur devrait gérer individuellement ses propres erreurs, ce qui entraînerait :

Une duplication importante du code

Des réponses API incohérentes selon les endpoints

Une maintenance plus complexe

Un risque d'exposition involontaire d'informations techniques sensibles

12.7.1. Classe métier `ApiError`

Une classe personnalisée `ApiError` a été créée **afin de représenter les erreurs métier contrôlées**.

Elle permet de transporter de manière structurée :

le code HTTP (`statusCode`)

un identifiant fonctionnel (`code`)

un message utilisateur (`message`)

des détails complémentaires éventuels de champs (`fields`)

Exemple :

```
throw new ApiError(
  404,
  "ARTISAN_NOT_FOUND",
  "Artisan non trouvé"
);
```

Cette approche est plus robuste qu'une erreur JavaScript standard (`Error`) car elle permet de transmettre des informations précises au middleware global. L'`ApiError` hérite du comportement standard de la classe JavaScript `Error`.

12.7.2. Middleware global de gestion des erreurs

Un **middleware Express** dédié (`apiErrorHandler`) centralise le traitement de toutes les erreurs de l'application.

Son rôle est :

- d'intercepter automatiquement les erreurs propagées via `next(error)`
- identifier les erreurs métier contrôlées
- normaliser le format des réponses JSON
- éviter la duplication de logique dans les contrôleurs

Format de réponse standard :

```
{
  "success": false,
  "error": {
    "message": "Données invalides.",
    "code": "VALIDATION_ERROR",
    "fields": {
      "email": "Format d'email invalide"
    }
  }
}
```

12.7.3. Protection des informations sensibles

En **environnement de développement**, la pile d'erreur (`stack`) est conservée **afin de faciliter le débogage**. En **environnement de production**, ces informations sont **volontairement masquées**.

Cela évite d'exposer :

- la structure interne du backend
- les chemins de fichiers serveur
- les dépendances utilisées
- des détails techniques exploitables par un attaquant

12.7.4. Cohérence frontend / backend

Le frontend exploite un format d'erreur unique grâce à une normalisation côté client via l'utilitaire `buildApiError()`.

Les erreurs API sont ainsi converties dans une structure homogène :

- `error.message`
- `error.code`
- `error.fields`

Cela permet :

l'affichage des erreurs de validation champ par champ ou groupées

la gestion des erreurs métier

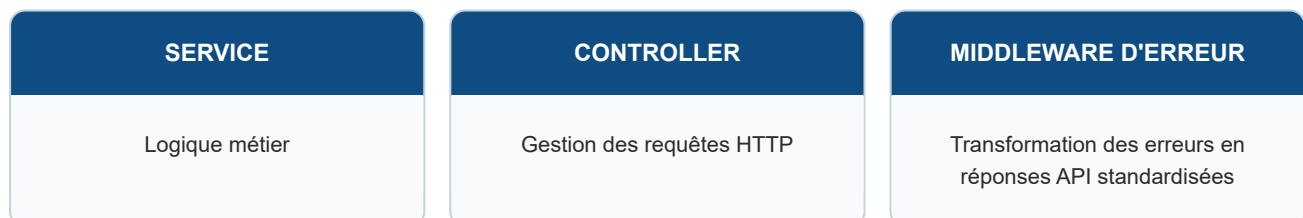
la prise en charge des limitations de requêtes (`rate limiting`)

Exemple :

Formulaire invalide	Affichage des erreurs par champ ou groupées
Artisan inexistant	Message métier explicite
Trop de tentatives	Message de limitation clair

12.7.5. Séparation des responsabilités

Cette architecture applique une séparation claire des responsabilités :



Cette organisation améliore :

La lisibilité du code

La maintenabilité

La réutilisabilité

13. Qualité du projet

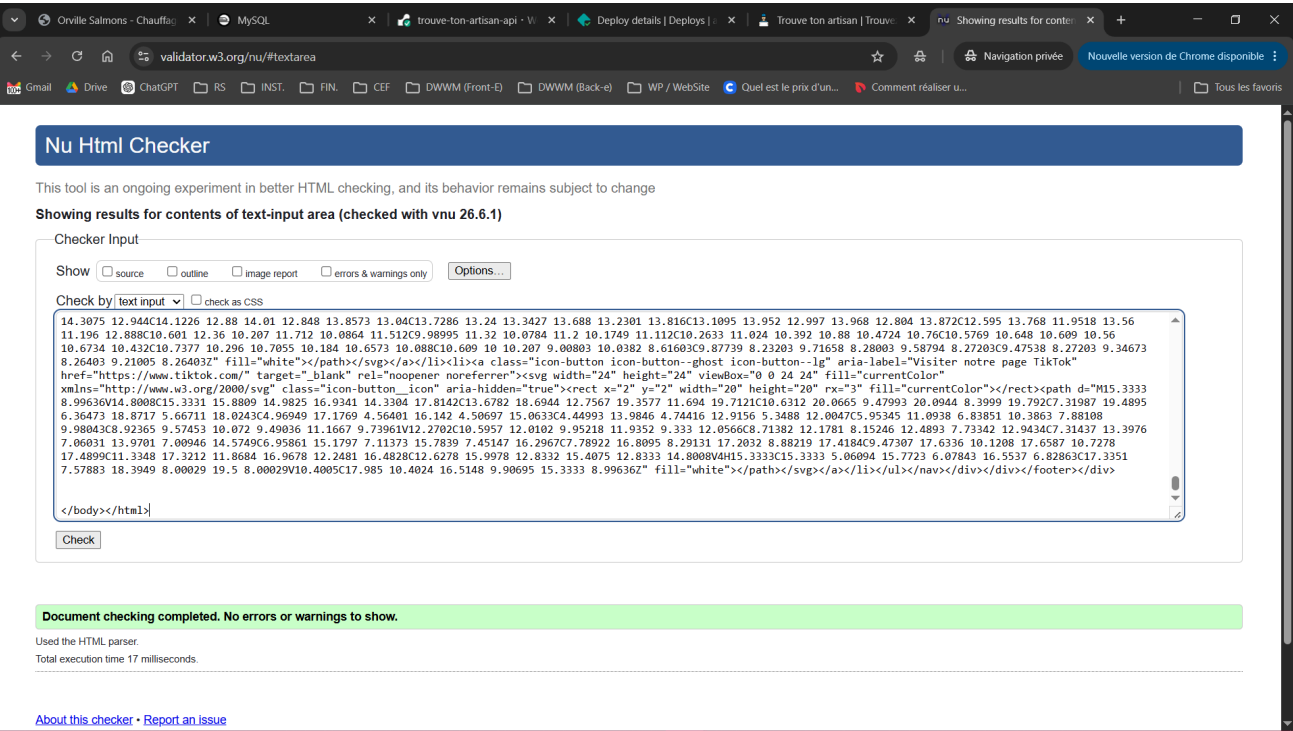
13.1. Validation W3C

Le projet a été validé à l'aide des outils du W3C afin de vérifier la conformité du code HTML généré.

Les principales pages du projet sont validées :

- Page d'accueil
- Liste des artisans
- Fiche artisan

Aucune erreur bloquante n'est détectée lors des contrôles.



The screenshot shows the Nu Html Checker website interface. At the top, there's a navigation bar with the site's name and a 'New version of Chrome available' notification. Below this is a header section with the title 'Nu Html Checker'. The main content area displays the message 'Document checking completed. No errors or warnings to show.' in a green box. Above this message, there's a section for 'Showing results for contents of text-input area (checked with vnu 26.6.1)'. This section includes a 'Checker Input' field with a 'Show' button and a 'Check by text input' dropdown menu. The input field contains a large block of HTML code, which appears to be a complex SVG element. Below the input field, there's a 'Check' button. The overall layout is clean and modern, with a white background and blue accents.

13.2. Audit Lighthouse

Des audits Lighthouse ont été réalisés afin de mesurer :

Les performances

L'accessibilité

Les bonnes pratiques

Le référencement

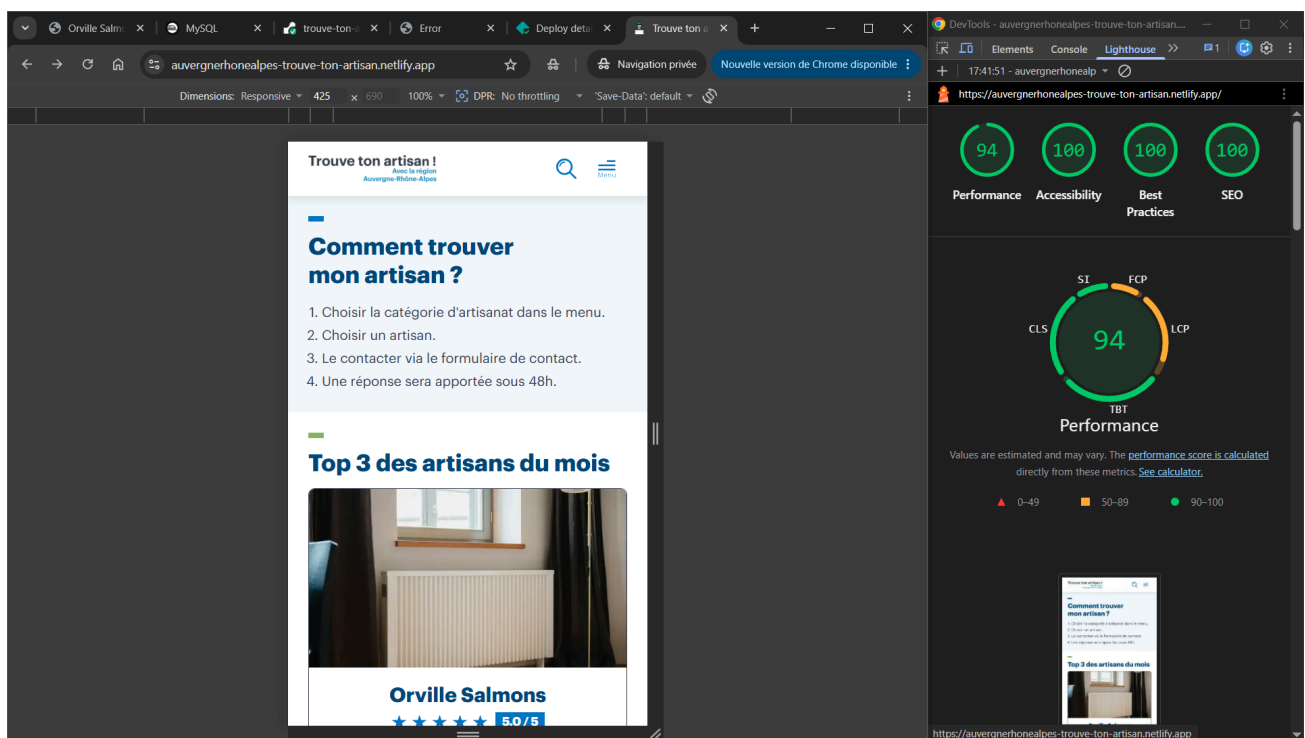
Les principales pages de l'application ont été auditées sur mobile et desktop :

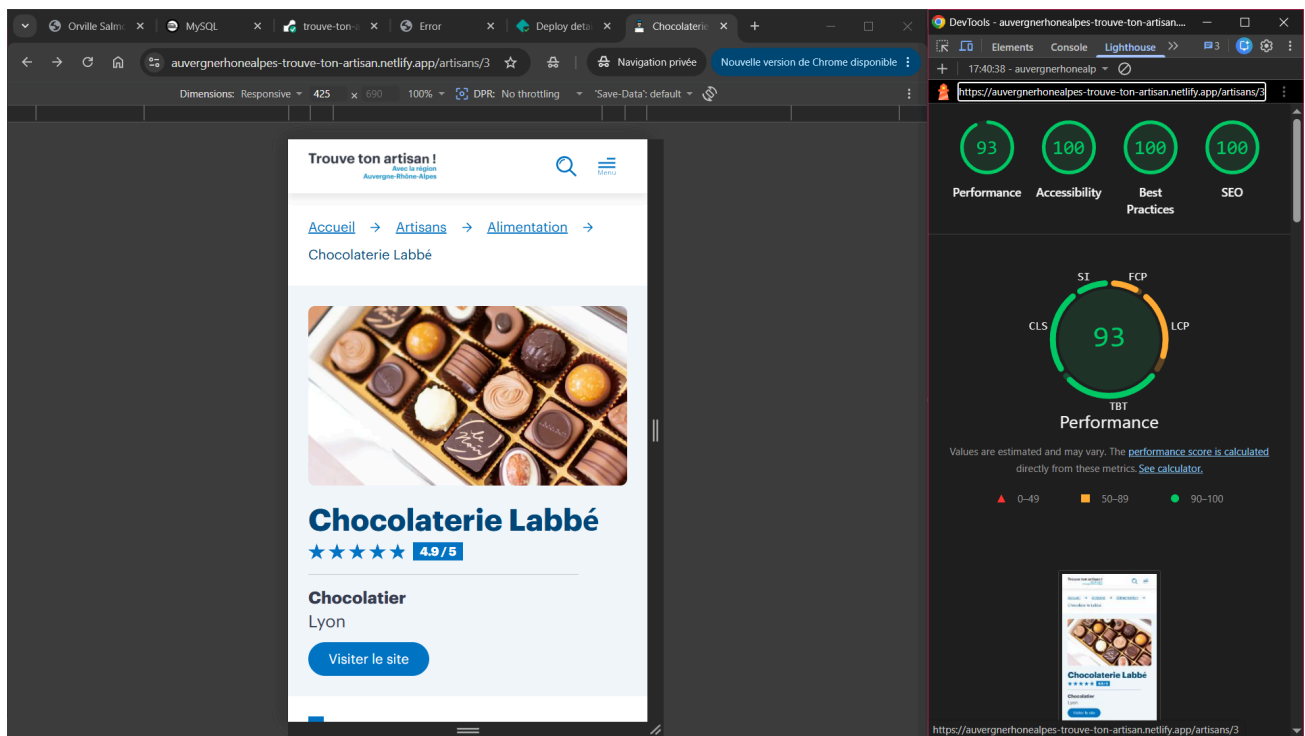
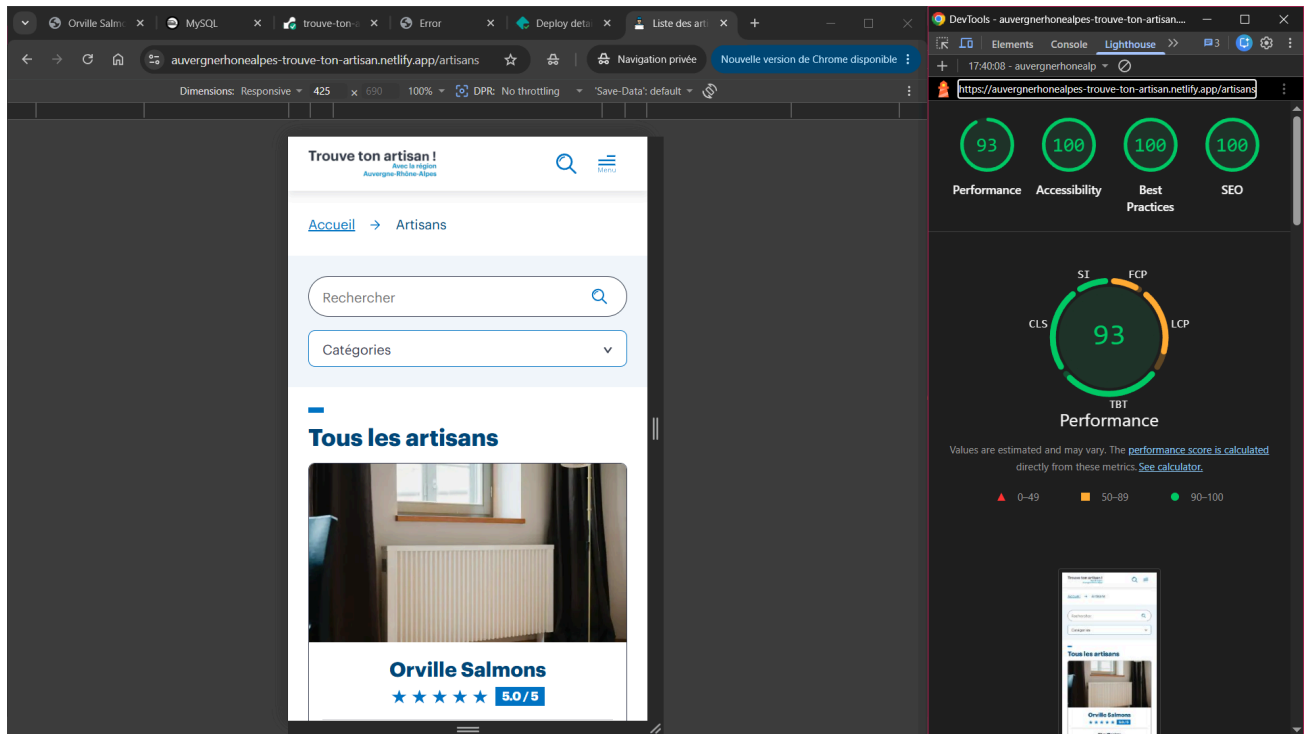
- Page d'accueil
- Liste des artisans
- Fiche artisan

Les résultats obtenus montrent :

- un bon niveau d'accessibilité
- des performances satisfaisantes
- une conformité SEO correct
- l'absence de problème majeur bloquant

AUDIT LIGHTHOUSE - MOBILE

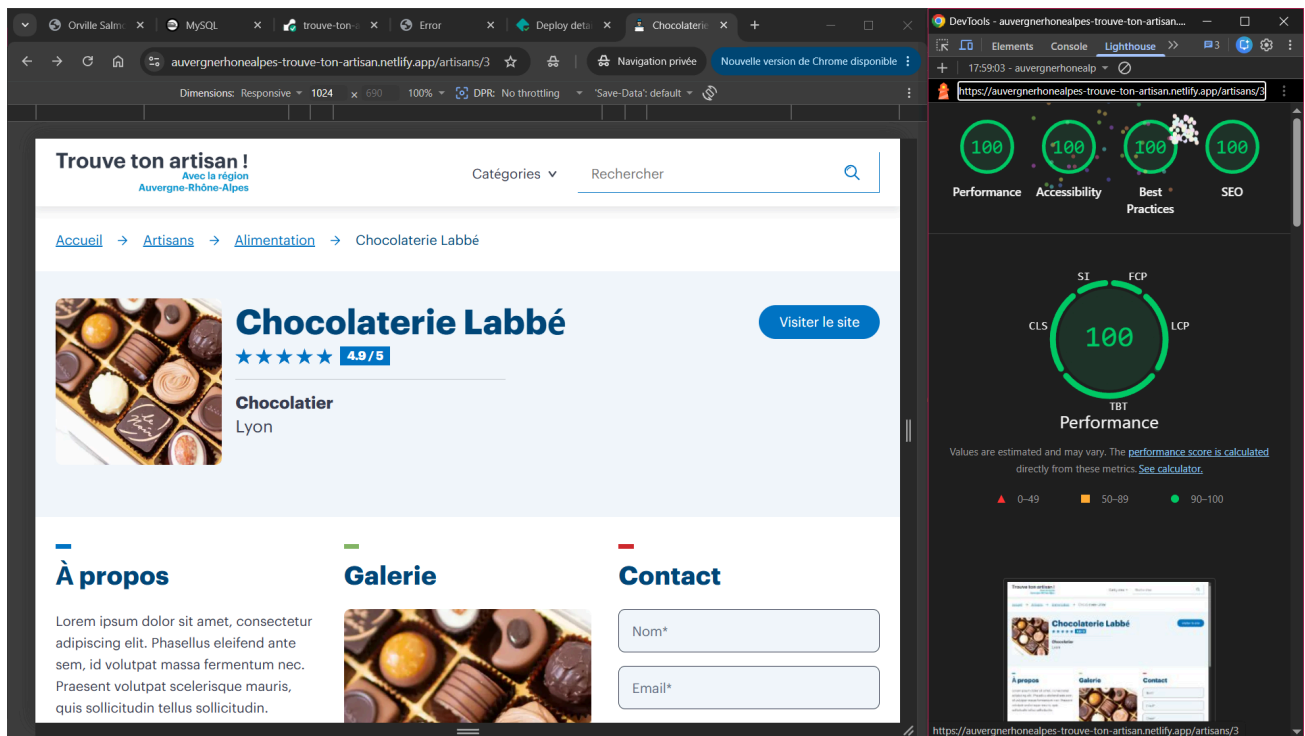




AUDIT LIGHTHOUSE - DESKTOP

The screenshot shows the homepage of 'Trouve ton artisan ! Avec la région Auvergne-Rhône-Alpes'. The page features a search bar, a list of categories, and a 'Top 3 des artisans du mois' section. The Lighthouse audit results are displayed on the right, showing scores of 100 for Performance, Accessibility, Best Practices, and SEO. The Performance section includes a circular chart with '100' in the center, and a list of metrics: ST, FCP, CLS, LCP, and TBT. The audit also includes a 'Performance' section with a 'TBT' metric and a 'Performance' section with a 'TBT' metric.

The screenshot shows the 'Tous les artisans' page of 'Trouve ton artisan ! Avec la région Auvergne-Rhône-Alpes'. The page features a search bar, a list of categories, and a 'Tous les artisans' section. The Lighthouse audit results are displayed on the right, showing scores of 100 for Performance, Accessibility, Best Practices, and SEO. The Performance section includes a circular chart with '100' in the center, and a list of metrics: ST, FCP, CLS, LCP, and TBT. The audit also includes a 'Performance' section with a 'TBT' metric and a 'Performance' section with a 'TBT' metric.



14. Liens du projet

Dépôt GitHub	https://github.com/pixseed/devoir-bilan-trouve-ton-artisan
Application en ligne	https://auvergnerhonealpes-trouve-ton-artisan.netlify.app/
API backend	https://trouve-ton-artisan-api-xt2w.onrender.com/